

Write a Program to Implement a Class STUDENT having following members:

Data members	
Member	Description
sname	Name of the student
Marks array	Marks of the student
total	Total marks obtained
Tmax	Total maximum marks

Member functions	
Member	Description
assign()	Assign Initial Values
compute()	to Compute Total, Average
display()	to Display the Data.

2. Write a program to demonstrate the i) Operator Overloading ii) Function Overloading.
3. Write a Program to demonstrate friend function and friend class.
4. Write a C++ Program that illustrates single inheritance.
5. Write a C++ Program that illustrates multiple inheritance.
6. Write a C++ Program that illustrate multilevel inheritance
7. To develop a C++ program to add constructors and destructor for Initializing and destroying objects.
8. To write a C++ program to use the THIS pointer
9. To write a C++ program to add a virtual function.
10. To write a C++ program to convert data between objects of different classes.
11. To write a C++ program to perform file operations.
12. To develop a C++ program to pass and return objects to and from a function.
13. To Write a C++ program to overload unary operator.
14. To write a C++ program to overload binary operator.

Some Example Codes

Aim: To write a C++ program to convert data between objects of different Classes. Data conversion in C++ includes conversions between basic types and user defined types, and conversions between different user-defined types.

```
#include <iostream>
#include <string>
using namespace std;
class time12
{
private:
    bool pm;
    int hrs;
```

```

        int mins;
public:
time12()
{
    pm=true;
    hrs=0;
    mins=0;
}
time12(bool ap, int h, int m)
{
    pm=ap;
    hrs=h;
    mins=m;
}
void display() const
{
    cout << hrs << ':';
    if(mins < 10)
        cout << '0';
    cout << mins << ' ';
    if(pm==true)
        cout<< "PM";
    else
        cout<<"AM";
}
};

class time24
{
private:
    int hours;
    int minutes;
    int seconds;
public:
time24()
{
    hours=0;
    minutes=0;
    seconds=0;
}
void entered_date(int h, int m, int s)

```

```

{
    hours=h;
    minutes=m;
    seconds=s;
}
void display() const
{
    if(hours < 10)
        cout << '0';
    cout << hours << ':';
    if(minutes < 10)
        cout << '0';
    cout << minutes << ':';
    if(seconds < 10)
        cout << '0';
    cout << seconds;
}
operator time12() const;
};

time24::operator time12() const
{
    int hrs24 = hours;
    //bool pm = hours < 12 ? false : true;
    //round secs
    bool pm;
    if(hours < 12)
        pm=false;
    else
        pm=true;
    int roundMins = seconds < 30 ? minutes : minutes+1;
    if(roundMins == 60)
    {
        roundMins=0;
        ++hrs24;
        if(hrs24 == 12 || hrs24 == 24)
            if(pm==true)
                pm=false;
            else
                pm=true;
    }
}

```

```

int hrs12;
if(hrs24 < 13)
    hrs12=hrs24;
else
    hrs12=hrs24-12;
if(hrs12==0)
{
    hrs12=12;
    pm=false;
}
return time12(pm, hrs12, roundMins);
}

```

```

int main()
{
    int h, m, s;
    time24 t24;
    cout<<"Default time is ";
    t24.display();
    while(true)
    {
        cout <<"\n\nEnter 24-hour time: \n";
        cout <<"Enter \nHours (0 to 23): ";
        cin >> h;
        if(h > 23)
            return(1);
        cout <<"Minutes: ";
        cin >> m;
        cout <<"Seconds: ";
        cin >> s;
        t24.entered_date(h, m, s);
        cout <<"You entered: ";
        t24.display();
        time12 t12 = t24;
        cout <<"\n12-hour time: ";
        t12.display();
        cout << "\n\n";
    }
    return 0;
}

```

```

#include <iostream>
using namespace std;

class Box {
public:
    double getVolume(void) {
        return length * breadth * height;
    }
    void setLength( double len ) {
        length = len;
    }
    void setBreadth( double bre ) {
        breadth = bre;
    }
    void setHeight( double hei ) {
        height = hei;
    }

    // Overload + operator to add two Box objects.
    Box operator+(const Box& b) {
        Box box;
        box.length = this->length + b.length;
        box.breadth = this->breadth + b.breadth;
        box.height = this->height + b.height;
        cout<<"\nAddress of Box is "<<&box;
        cout<<"\nAddress of b is "<<&b;
        cout<<"\nAddress of this is "<<this;
        return box;
    }

    double length;    // Length of a box
    double breadth;   // Breadth of a box
    double height;    // Height of a box
};

```

```

// Main function for the program
int main() {
    Box Box1;          // Declare Box1 of type Box
    Box Box2;          // Declare Box2 of type Box
    Box Box3;          // Declare Box3 of type Box
    double volume = 0.0; // Store the volume of a box here
    cout<<"\nAddress of Box1 is "<<&Box1;
    cout<<"\nAddress of Box2 is "<<&Box2;
    cout<<"\nAddress of Box3 is "<<&Box3;
    // box 1 specification
    Box1.setLength(6.0);
    Box1.setBreadth(7.0);
    Box1.setHeight(5.0);

    // box 2 specification
    Box2.setLength(12.0);
    Box2.setBreadth(13.0);
    Box2.setHeight(10.0);

    // volume of box 1
    volume = Box1.getVolume();
    cout << "\nVolume of Box1 : " << volume <<endl;

    // volume of box 2
    volume = Box2.getVolume();
    cout << "Volume of Box2 : " << volume <<endl;

    // Add two object as follows:
    Box3 = Box1 + Box2;
    cout<<"\n\nBox1 Length is "<<Box1.length;
    cout<<"\nBox1 Breadth is "<<Box1.breadth;
    cout<<"\nBox1 Height is "<<Box1.height;

    cout<<"\n\nBox2 Length is "<<Box2.length;
    cout<<"\nBox2 Breadth is "<<Box2.breadth;
    cout<<"\nBox2 Height is "<<Box2.height;

    cout<<"\n\nBox3 Length is "<<Box3.length;
    cout<<"\nBox3 Breadth is "<<Box3.breadth;
    cout<<"\nBox3 Height is "<<Box3.height;
    // volume of box 3

```

```

    volume = Box3.getVolume();
    cout << "\nVolume of Box3 : " << volume << endl;

    return 0;
}

```

```

#include <iostream>
using namespace std;

```

```

class Base {
public:
    virtual void print() {
        cout << "Base Function" << endl;
    }
};

```

```

class Derived : public Base {
public:
    void print() override {
        cout << "Derived Function" << endl;
    }
};

```

```

int main() {
    Derived derived1;

    // pointer of Base type that points to derived1
    Base* base1 = &derived1;

    // calls member function of Derived class
    base1->print();

    return 0;
}

```

```

#include<iostream>

```

```

using namespace std;
class emp
{
    private:
        int id;
        char name[10];
        int salary;
        void get_data()
        {
            cout<<"Enter Id,Name and Salary of employee"<<endl;
            cin>>id>>name>>salary;
        }
    public:
        void disp()
        {
            get_data();
            cout<<"\n\nDetails are"<<endl;
            cout<<"Emp ID "<< id<<endl<<"Emp Name " <<name <<endl<<"Emp Salary "<<salary;
        }
};

int main()
{
    emp obj;
    obj.disp();
}

```

// C++ program to demonstrate function overriding

```

#include <iostream>
using namespace std;

class Base {
    public:
        virtual void print() {
            cout << "Base Function" << endl;
        }
};

class Derived : public Base {
    public:

```

```
void print() {  
    cout << "Derived Function" << endl;  
}  
};
```

```
int main() {  
    Derived derived1;  
    //derived1.print();  
    Base* base1 = &derived1;  
    base1->print();  
    return 0;  
}
```

```
// C++ program to demonstrate  
// protected access modifier  
#include <bits/stdc++.h>  
using namespace std;
```

```
// base class  
class Parent  
{  
    // protected data members  
    protected:  
    int id_protected;  
  
};
```

```
// sub class or derived class from public base class  
class Child: public Parent  
{  
    public:  
    void setId(int id)  
    {  
  
        // Child class is able to access the inherited  
        // protected data members of base class  
  
        id_protected = id;  
  
    }  
}
```

```

void displayId()
{
    cout << "id_protected is: " << id_protected << endl;
}
};

// main function
int main() {

    Child obj1;

    // member function of the derived class can
    // access the protected data members of the base class

    obj1.setId(81);
    obj1.displayId();
    return 0;
}

```

```

#include <iostream>
using namespace std;

class Student
{
public:
    double marks;
    Student(double m)
    {
        marks=m;
    }
};

double average_marks(Student s1, Student s2)
{
    return (s1.marks + s2.marks)/2 ;
}

int main()

```

```

{
    Student student1(88.0), student2(56.0);
    cout << "Average Marks = " << average_marks(student1, student2) << "\n";
    return 0;
}

```

```

-----
---
-----
---
```

```

#include <stdio.h>
void sum(int a, int b);
void sum(int a, int b, int c);
void sum(float a, int b);
void sum(int a, float b);
void sum(float a, float b);
void sum(char a, char b);
int main()
{
    printf("Hello World");

```

```

    sum(x,y);
    printf();
    return 0;
}

```

```

void sum(int a, int b)
{
    sum=a+b;
}

```

```

void sum(float a, int b)
{
    sum=a+b;
}

}

```

```
---

#include <iostream>
using namespace std;

class Box {
public:
    double getVolume(void) {
        return length * breadth * height;
    }
    void setLength( double len ) {
        length = len;
    }
    void setBreadth( double bre ) {
        breadth = bre;
    }
    void setHeight( double hei ) {
        height = hei;
    }

    // Overload + operator to add two Box objects.
    Box operator+(const Box& b) {
        Box box;
        box.length = this->length + b.length;
        box.breadth = this->breadth + b.breadth;
        box.height = this->height + b.height;
        cout<<"\nAddress of Box is "<<&box;
        cout<<"\nAddress of b is "<<&b;
        cout<<"\nAddress of this is "<<this;
        return box;
    }

    double length;    // Length of a box
    double breadth;   // Breadth of a box
    double height;    // Height of a box
};
```

